The Detection Pipeline — Step by Step

Imagine you have a video stream. In the first frame, someone draws a box around an object (say a car). Your job: **in every subsequent frame, find where that car moved to.** That's visual object tracking.

The naive approach: slide the template image across every possible position in the new frame, compare pixel-by-pixel, find the best match. This is **incredibly slow** — for a 32×32 patch, you'd need to check thousands of positions, each requiring 1024 comparisons.

KCF's trick: Instead of sliding and comparing in the spatial domain, transform everything into the **frequency domain** using FFT. In the frequency domain, the entire sliding-and-comparing operation collapses into a single element-wise multiply. That's the fundamental insight.

Step 1: Hann Windowing

What: Multiply the input patch by a bell-shaped curve that's 1.0 in the centre and tapers to 0.0 at the edges.

Why: The FFT assumes the signal repeats infinitely. If your patch has value 0.8 at the left edge and 0.1 at the right edge, the FFT "sees" a sharp jump where they wrap around. This creates fake high-frequency noise called **spectral leakage**. The Hann window smoothly forces all edges to zero, eliminating the jump.

In hardware: Two ROM lookups (one for row, one for column), multiply them together, then multiply with the pixel value. All done in Q8.8 fixed-point. One pixel per clock cycle, 1024 cycles total.

Step 2: 2D FFT (Forward Transform)

What FFT does in plain English: It takes a signal (our image patch) and decomposes it into a sum of sine and cosine waves at different frequencies. The output tells you "how much" of each frequency is present, and at what phase.

Think of it like a prism splitting white light into a rainbow — the FFT splits a signal into its frequency components.

Why we need it: In the frequency domain, convolution (sliding comparison) becomes simple multiplication. This is the key mathematical identity:

```
None
Convolution in space  ↔  Multiplication in frequency
```

So instead of sliding a template across 1024 positions (1 million operations), we do ONE FFT, ONE multiply, ONE IFFT.

How the 2D FFT works — row-column decomposition:

1. Apply a 1D FFT to each of the 32 rows (32 separate transforms)
2. Apply a 1D FFT to each of the 32 columns of the result (32 more transforms)

That's it. A 2D transform decomposes into 64 1D transforms.

How the 1D FFT works — the Cooley-Tukey algorithm:

The direct DFT formula for N points requires N^2 multiplications. The FFT reduces this to $N \cdot \log_2(N)$ by exploiting symmetry.

For N=32 ($\log_2(32) = 5$ stages):

```
None
Stage 0: pairs separated by 1  → 16 butterflies
Stage 1: pairs separated by 2  → 16 butterflies
Stage 2: pairs separated by 4  → 16 butterflies
Stage 3: pairs separated by 8  → 16 butterflies
Stage 4: pairs separated by 16 → 16 butterflies
Total: 80 butterfly operations
```

Before the butterflies start, the inputs are reordered using **bit-reversal permutation**. Index 3 (binary `00011`) maps to index 24 (binary `11000` — the bits reversed). This reordering ensures the butterfly pairs line up correctly at each stage.

Step 3: The Butterfly Unit

What: The atomic operation of the FFT. It takes two complex numbers `a` and `b`, multiplies `b` by a twiddle factor `W`, then computes:

```
None
output_top    = a + W·b
output_bottom = a - W·b
```

Why "butterfly"? If you draw the signal flow diagram, the crossing wires look like butterfly wings.

What are twiddle factors? They're pre-computed complex rotation values:

```
None
W₃₂ᵏ = cos(2πk/32) - j·sin(2πk/32)
```

Think of them as "how much to rotate" the data at each step. They're stored in ROM (16 values — symmetry means we only need half).

Scaled butterfly: Each butterfly divides its outputs by 2 (arithmetic right shift `>>>1`). After 5 stages, total division = $2^5 = 32 = N$. This prevents numbers from growing too large and overflowing the 16-bit registers.

Step 4: Element-wise Conjugate Multiply

What: For each of the 1024 frequency bins, compute:

```
None
result[k] = conj(a^[k]) × X^[k]
```

What is `conj` (conjugate)? A complex number `a + jb` has conjugate `a - jb`. You just flip the sign of the imaginary part.

What is `α̂` (alpha hat)? The pre-trained filter coefficients. During training (done offline in Python), the algorithm learns WHAT the target looks like by computing:

```
None
α̂ = Ŷ / (|X̂|² + λ)
```

where Ŷ is the desired Gaussian response (a bell curve centered where the target is) and X̂ is the FFT of the training patch. The small λ prevents division by zero.

What does the multiply accomplish? In the spatial domain, this would be a full sliding convolution of the filter with the image. In frequency domain, it's just 1024 independent multiplies. Each multiply takes one clock cycle.

The `cconj_mul` module: Wraps a regular complex multiplier (`cmul`) but negates the imaginary part of one input to conjugate it. The complex multiply formula:

```
None
(a_re + j·a_im) × (b_re - j·b_im) =
    real: a_re·b_re + a_im·b_im
    imag: a_im·b_re - a_re·b_im
```

Step 5: 2D IFFT (Inverse Transform)

What: Transforms the frequency-domain product back to the spatial domain, producing the **response map** — a 32×32 grid where brighter values indicate "the target is more likely here."

The conjugate trick: Instead of building a separate IFFT module, we reuse the FFT:

```
None
IFFT(X) = conj(FFT(conj(X))) / N²
```

Steps: negate all imaginary parts → run normal FFT → negate all imaginary parts again. The division by N² comes for free from the scaled butterfly.

Step 6: Peak Finder

What: Scans all 1024 values of the response map, keeps track of the maximum value and its position. After 1024 cycles, outputs the (row, col) of the peak.

What the peak means: If the peak is at (18, 19), it means the target's centre is at row 18, column 19 within the patch. In a real tracker, this displacement would update the bounding box position.

Fixed-Point Arithmetic (Q8.8)

Why not floating point? Floating-point hardware (IEEE 754) requires massive multipliers, adders, and special-case logic. A single FP multiply can take hundreds of LUTs on an FPGA. Fixed-point uses regular integer arithmetic.

Q8.8 format: A 16-bit signed integer where:

- Top 8 bits = integer part (-128 to +127)
- Bottom 8 bits = fractional part (resolution 1/256 ≈ 0.004)
- To convert: `float_value × 256 = raw_integer`
- Example: 0.1 × 256 = 25.6 → stored as 26 → actual value 26/256 = 0.1016

Multiplication: Two Q8.8 values multiplied give Q16.16 (32 bits). To get back to Q8.8, extract bits [23:8] — this keeps 8 integer bits and 8 fractional bits from the middle of the 32-bit product.

The Scaling Bug — The Hardest Problem

This is the most important debugging story for your presentation.

The problem: The pipeline completed but always output the wrong peak location with a peak value of 0 or 1.

Root cause analysis:

The 2D FFT applies `>>>1` at each butterfly stage. For 1D: 5 stages = ÷32 = ÷N. For 2D (row FFT then column FFT): ÷N × ÷N = ÷N² = ÷1024.

The IFFT also uses the FFT internally (conjugate trick), applying another ÷N².

So the total pipeline scaling was: **÷N² from FFT × ÷N² from IFFT = ÷N⁴ = ÷1,048,576**.

The actual response peak in Python is 0.142. In hardware: 0.142 ÷ 1,048,576 ≈ 0.000000135. In Q8.8 that rounds to **zero**. All signal was lost to quantization.

The fix (two-part):

1. **Asymmetric FFT scaling:** The forward FFT scales only the row pass (÷N) but leaves the column pass unscaled. This was done by making `scale_en` a **runtime signal** instead of a compile-time parameter, so the same FFT hardware can switch scaling on/off between row and column passes. Total forward FFT scaling: ÷N instead of ÷N².
2. **Alpha pre-scaling:** The filter coefficients are multiplied by K=16 in Python before quantization. This compensates for the remaining ÷N factor, giving final output = (K/N) × response = 0.5 × response ≈ 0.071 → Q8.8 raw value of 18. Distinguishable from noise.

Why K=16 and not K=32? With K=32, some alpha values would exceed 127 (Q8.8 max), and the complex multiplier output could overflow. K=16 keeps max|α×16| ≈ 71.7, and the product magnitude ≈ 79, safely below 128.

The Spatial Reversal Bug

The problem: After fixing scaling, the peak appeared at (14, 13) instead of the expected (18, 19).

The insight: 32 - 18 = 14, and 32 - 19 = 13. The response was **spatially reversed**.

Root cause: The `cconj_mul` module computes `a × conj(b)`. We were passing `alpha` as `a` and `X_hat` as `b`, computing `α × conj(X̂)`. But the math requires `conj(α) × X̂`.

These are NOT the same:

- `conj(a) × X̂` → IFFT gives `response(n)`
- `a × conj(X̂)` → IFFT gives `response(-n mod N)` (spatially reversed!)

The fix: Swap the two inputs — pass `X_hat` as `a` (not conjugated) and `alpha` as `b` (conjugated by the module). One line change, from wrong peak to perfect match.

Key Points for Your Presentation

Architecture highlights:

- Complete end-to-end detection pipeline in synthesizable SystemVerilog
- 19,300 cycles per detection at 100 MHz = 193 µs latency
- Single-butterfly iterative FFT architecture (area-efficient, one multiplier reused 80 times)
- Row-column decomposition turns 2D FFT into 64 × 1D FFTs
- IFFT reuses FFT hardware via conjugate trick (zero additional multipliers)
- All coefficients (Hann window, twiddle factors, alpha) pre-computed and stored in ROM

Smart engineering decisions:

- Q8.8 fixed-point keeps the entire design in 16-bit datapaths — no floating point
- Runtime `scale_en` signal allows one FFT module to operate in both scaled and unscaled modes
- Asymmetric row/column scaling preserves signal magnitude while preventing overflow
- Alpha pre-scaling by K=16 compensates for FFT attenuation without overflowing the multiplier
- Python golden reference script generates all test data AND verifies correctness

Bug fixes worth mentioning:

1. **Counter overflow (Bug C):** A 10-bit counter compared against 1024, but 10 bits max out at 1023 — the comparison was always false, causing an infinite loop. Fixed by widening to 11 bits.
2. **Timing misalignment (Bugs A/B):** Hann ROM addresses and multiply inputs were registered (delayed by one cycle), causing off-by-one alignment errors. Fixed by making them combinational wires.
3. **Scaling collapse:** Total ÷N⁴ attenuation destroyed all signal. Fixed with asymmetric FFT scaling + alpha pre-scaling.
4. **Spatial reversal:** Wrong conjugation order in complex multiply reversed the response map. Fixed by swapping operands.